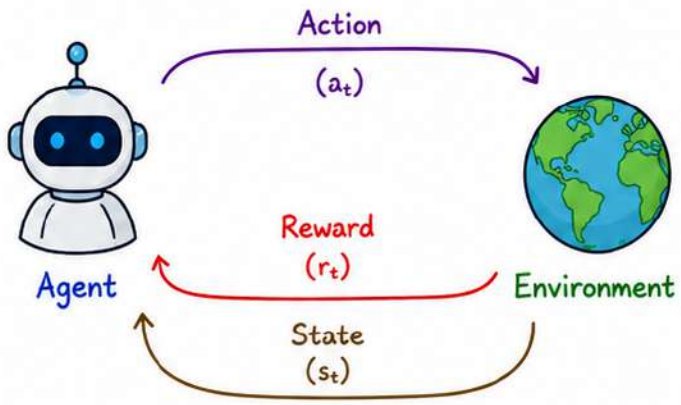
What is Reinforcement Learning?

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment to maximize cumulative reward.

Key Idea:

- The agent takes actions in an environment.
- The environment gives feedback in the form of a reward.
- The agent learns a policy (a mapping from states to actions) that maximizes total reward over time.

Reinforcement Learning Loop

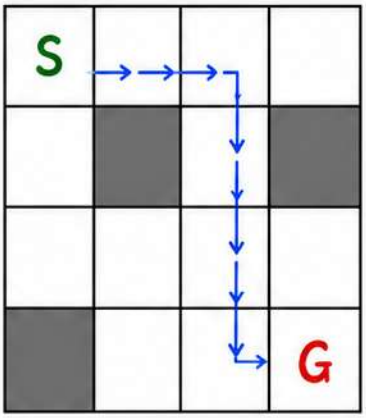


Goal:

Learn the best policy π^* that tells the agent what action to take in each state to maximize the total (cumulative) reward.

Example: Robot Navigation

A robot moves in a grid to reach the goal (G) from the start (S) while avoiding obstacles.



Actions:
Up, Down, Left, Right

Reward:
- +10 for reaching Goal
- -1 for each step
- -10 for hitting obstacle

Objective:
Reach the goal with maximum total reward (minimum steps).

Real-world Applications:


Game Playing (e.g., AlphaGo)


Robotics


Autonomous Driving


Finance


Healthcare

and more...

Reinforcement Learning (RL)

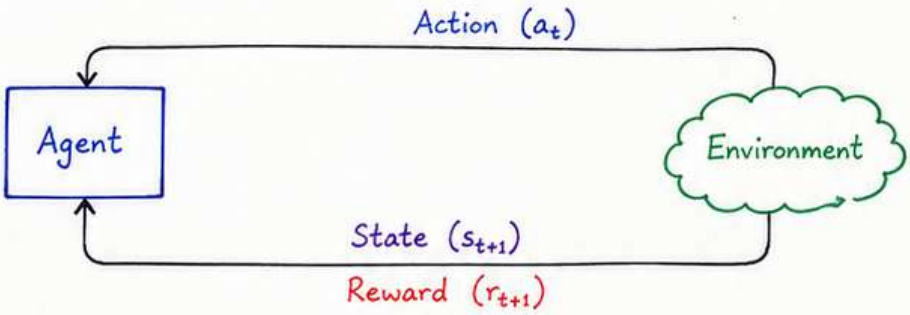
1. What is Reinforcement Learning?

Reinforcement Learning (RL) is a type of machine learning in which an agent learns to make decisions by interacting with an environment to maximize cumulative reward.

2. Key Components

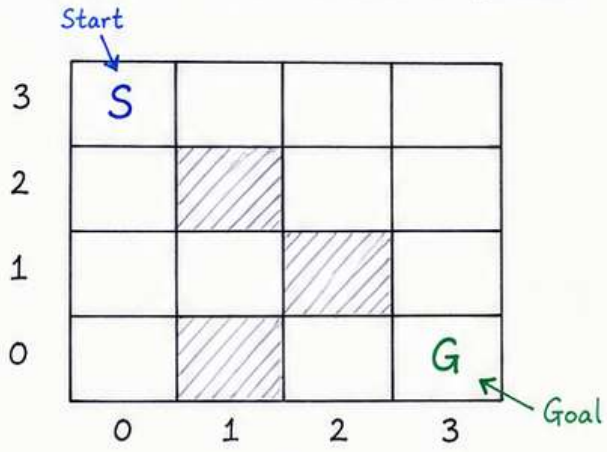
- Agent : The learner or decision maker.
- Environment : The surroundings with which the agent interacts.
- State (s) : The current situation of the environment.
- Action (a) : The choices available to the agent.
- Reward (r) : Feedback signal from the environment after an action.
- Policy (π) : The strategy or mapping from states to actions.

3. Reinforcement Learning Loop



The agent takes an action, the environment moves to a new state and gives a reward. The agent learns to choose actions that maximize the total (cumulative) reward over time.

4. Example: Grid World Navigation



The agent (S) wants to reach the goal (G) from the start while avoiding obstacles (shaded cells).

Actions	:	Up, Down, Left, Right
Rewards	:	+10 for reaching Goal
		−1 for each step taken
		−10 for hitting an obstacle

Objective:
Learn a policy that tells the agent which action to take in each state to reach the goal with maximum cumulative reward in minimum number of steps.

5. Real-World Applications

Game Playing (e.g., AlphaGo) | Robotics | Autonomous Driving | Finance | Healthcare

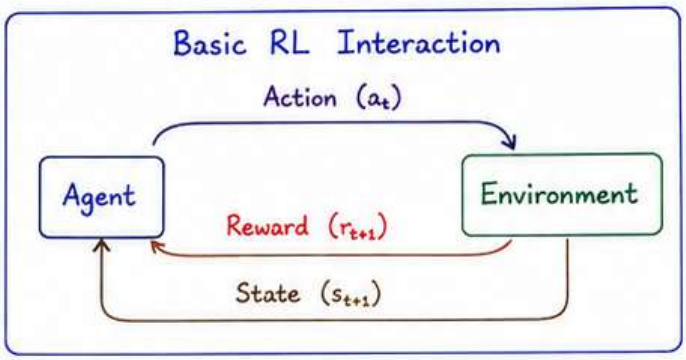
and more...

In short: RL is about learning by doing.
The agent improves through trial and error to achieve the best outcome.

Reinforcement Learning (RL)

1. Why RL?

- Many real-world problems involve sequential decision making where the outcome is not known in advance.
- RL allows an agent to learn by interacting with the environment and improve its performance over time.
- It is suitable for problems with delayed rewards and no explicit supervision.
- RL has wide applications in robotics, games, recommendation systems, resource management, autonomous driving, etc.



State (s) : Current situation of the environment
Action (a) : Choice made by the agent
Reward (r) : Feedback received after action
Policy (π) : Mapping from states to actions

2. Key Features of RL

★ Sequential Decision Making : Agent makes a sequence of decisions over time.
★ Interaction with Environment : Learning is achieved by trial and error through interaction.
★ Reward Signal : Learning is guided by rewards (positive or negative).
★ No Explicit Supervision : No labeled input-output pairs; feedback is in the form of rewards.
★ Balance of Exploration & Exploitation : Agent must explore new actions and exploit known good actions.
★ Delayed Rewards : The effect of an action may be seen after many steps.
★ Goal-Oriented : The objective is to learn a policy that maximizes cumulative reward.

3. History of RL (A Brief Timeline)

→ 1950s : Foundations in control theory and dynamic programming (Bellman, 1957).
→ 1960s-70s : Early work on Markov Decision Processes (MDP) and adaptive control.
→ 1980s : TD(0) learning (Sutton, 1988), Q-learning (Watkins, 1989).
→ 1990s : Function approximation, policy gradients, and actor-critic methods.
→ 2000s : RL applied to robotics, games, and real-world problems.
→ 2010s : Deep Reinforcement Learning (DRL) with DQN (2013), AlphaGo (2016) – major breakthrough.
→ 2020s : RL used in healthcare, finance, autonomous systems, and large language model alignment (RLHF).

> "From simple tables to deep neural networks, RL has evolved to solve complex real-world tasks."

4. Real-time Analogy to RL

Example: Learning to ride a bicycle

State: Your current balance, speed, position, road condition, etc.
→ **Action**: Pedal, turn handle, apply brake, change balance, etc.
→ **Reward**: Stay balanced and move ahead (+ve reward) Fall down (-ve reward)
→ **Learning**: You learn which actions keep you moving and avoid falling.
→ **Goal**: Ride the bicycle smoothly to reach the destination.

Through practice and feedback, you improve over time – that is Reinforcement Learning!

In short: RL is about learning by interacting with the environment, receiving rewards, and improving decisions to achieve the best possible outcome. ☺

Reinforcement Learning (RL)

1. Elements of RL

An RL problem is typically defined by the following key elements:

① Agent : The learner or decision maker that takes actions.

② Environment : Everything outside the agent with which it interacts.

③ State (s) : The current situation or configuration of the environment.
State space (S) = set of all possible states.

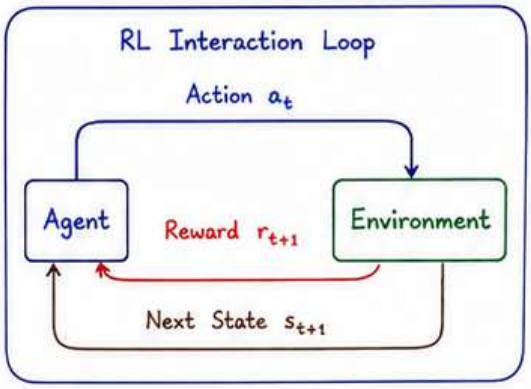
④ Action (a) : Choices available to the agent in a state.
Action space (A) = set of all possible actions.

⑤ Reward (r) : Immediate feedback received after taking an action.
It can be positive, negative or zero.

⑥ Policy (π) : The strategy or mapping from states to actions.
$\pi : S \rightarrow A$

⑦ Episode : A sequence of interactions from start to terminal state.

⑧ Goal : Learn a policy that maximizes the cumulative (total) reward.



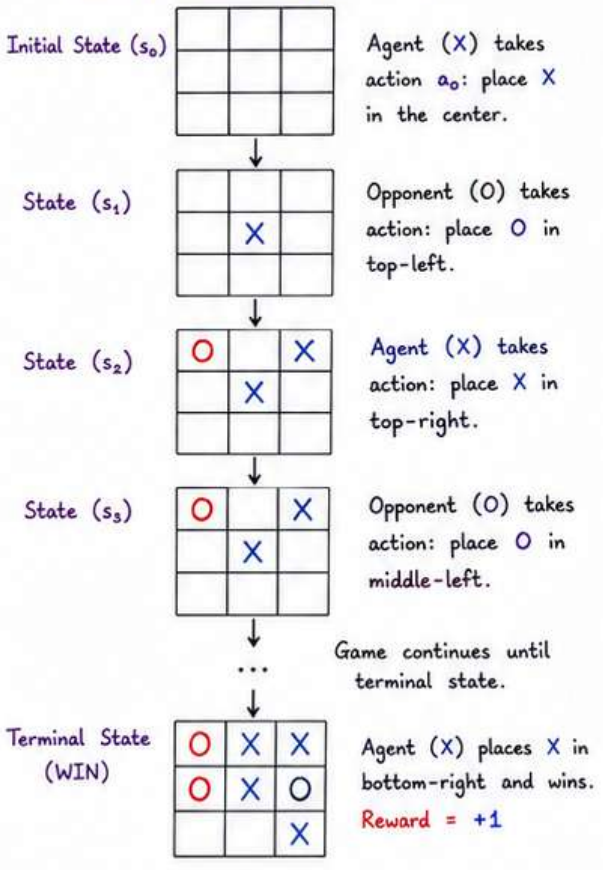
At time t, the agent is in state s_t.
It takes action a_t.
The environment moves to state s_{t+1} and gives reward r_{t+1}.
This process repeats until the episode ends.

2. Elements of RL and Tic-Tac-Toe Example

Consider a Tic-Tac-Toe game where the agent (X) tries to win against an opponent (O).

RL Element	Meaning	Tic-Tac-Toe Example
① Agent	The learner/decision maker.	The player 'X'.
② Environment	Everything the agent interacts with.	The game board and the opponent 'O'.
③ State (s)	Current situation of the environment.	Current board configuration. (There are 3^9 = 19,683 possible states)
④ Action (a)	Choices available to the agent.	Placing 'X' in any one of the empty cells.
⑤ Reward (r)	Feedback after an action.	+1 : Agent wins 0 : Draw -1 : Agent loses
⑥ Policy (π)	Strategy for choosing actions.	Best move selection strategy (e.g., always block opponent's win, try to win, etc.)
⑦ Episode	A complete game.	From empty board to win/draw/loss.
⑧ Goal	Maximize cumulative reward.	Win the game (get +1).

Example Game (Agent = X)



In RL, the agent learns by trial and error: it interacts with the environment, receives rewards, and improves its policy to achieve the goal (maximum cumulative reward).

→ Key Idea: Learn the best strategy (policy) so that good actions are chosen in every state to get the highest total reward.

Reinforcement Learning (RL) Terminologies

RL is a type of machine learning where an agent learns to make decisions by interacting with an environment so as to maximize cumulative reward.

1. Key RL Terminologies

① Time Step (t) : Discrete time index at which the agent interacts with the environment.
$t = 0, 1, 2, \ldots, T-1$.

② State (s_t) : The current situation of the environment at time t. It contains all information that is relevant for decision making.

③ Action (a_t) : The choice made by the agent from the set of all possible actions A.

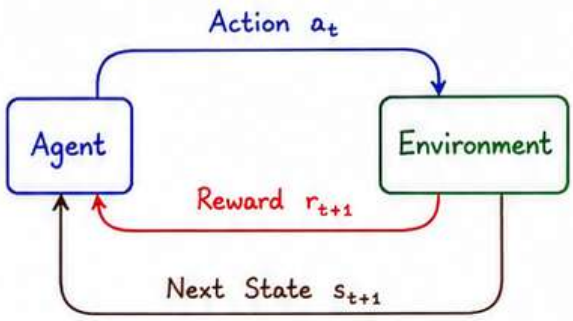
④ Reward (r_{t+1}) : Immediate feedback (scalar) received after taking action a_t in state s_t and moving to next state s_{t+1}. It can be positive, negative or zero.

⑤ Observation (o_t) : What the agent actually perceives about the environment.
$o_t = s_t$ in fully observable settings.
o_t gives partial information about s_t in partially observable settings.

⑥ Model : The (possibly unknown) dynamics of the environment.
It describes how next state and reward are generated.
Model is $P(s_{t+1}, r_{t+1} \mid s_t, a_t)$.

⑦ Policy (π) : The strategy or mapping followed by the agent to choose actions.
$\pi(a|s)$ = probability of taking action a in state s.

RL Interaction Loop



Typical Notation Summary

- t : time step
- s_t : state at time t
- a_t : action at time t
- r_{t+1} : reward received after taking a_t
- o_t : observation at time t
- s_{t+1} : next state
- π : policy
- γ : discount factor, $0 \le \gamma < 1$ (future rewards are discounted)

RL Objective

Learn a policy π^* that maximizes expected cumulative (discounted) reward

$$J(\pi) = E_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \right], \quad 0 \le \gamma < 1$$

2. RL Formulation Examples

Example	States (S)	Actions (A)	Rewards (R)	Goal / Objective	Typical Policy
(1) Grid World Navigation	All possible positions of the agent in the grid (i, j).	Up, Down, Left, Right	+10 for reaching goal −1 for each step −10 for hitting obstacle	Reach the goal with maximum cumulative reward in minimum steps.	Move towards goal while avoiding obstacles.
(2) Robotic Arm (Reaching)	Joint angles of arm / end-effector position.	Continuous torques applied to joints.	+1 for reaching target position, −1 for error otherwise.	Move end-effector to target position accurately.	Apply torques to minimize distance to target.
(3) Stock Trading	Market indicators (price, RSI, volume, holdings, cash, etc.)	Buy, Sell, Hold	Change in portfolio value (after transaction cost).	Maximize total profit over time.	Buy low, sell high based on learned strategy.
(4) Personalized Recommendation	User features, item features, interaction history.	Recommend item i from catalog.	+1 if user clicks/buys 0 otherwise.	Maximize long-term user engagement (clicks/likes/purchases).	Recommend items most likely to engage user.

1. Exploration vs Exploitation

- In Reinforcement Learning, the agent must decide whether to try new actions (explore) to gain more information, or choose the best known action (exploit) to get the highest reward.

Trade-off:
Too much exploration → slow learning (may get less reward in short term)
Too much exploitation → may miss better actions
Good RL balances both to maximize long-term reward.

★ **Exploration (Search)**

Try new or uncertain actions to discover better states and rewards.

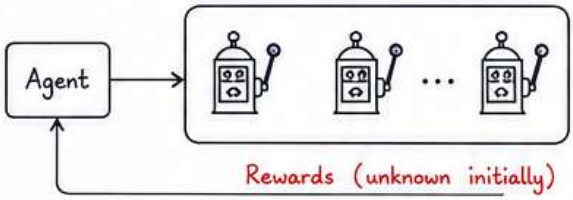
Methods: Random action selection, ε-greedy (choose random action with probability ε), Softmax action selection, etc.

★ **Exploitation (Utilization)**

Choose the action that is currently known to give the highest reward.

Methods: Greedy action selection, Follow the current best policy, Use learned value / Q estimates.

Example: Multi-armed Bandit Problem



The agent does not know which arm (action) gives the highest payout (reward).
- Exploration: Try different arms to learn their payouts.
- Exploitation: Pull the arm that seems best (highest estimated reward).

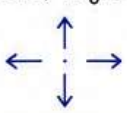
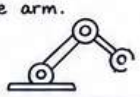
2. Formulation of RL Agents for Few Examples

An RL problem is typically formulated as an MDP: (S, A, P, R, γ)

- S : Set of states
- A : Set of actions
- P : State transition probability $P(s' \mid s, a)$
- R : Reward function $R(s, a)$ or $R(s, a, s')$
- γ : Discount factor $(0 \le \gamma < 1)$

Objective:
Find a policy $\pi(a|s)$ that maximizes expected cumulative (discounted) reward:

$$J(\pi) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \right]$$

Example	States (S)	Actions (A)	Rewards (R)	Goal / Objective	Typical Policy
(1) Grid World Navigation	All possible positions in the grid (i,j). 	Up, Down, Left, Right 	+10 for reaching goal −1 for each step −10 for hitting obstacle	Reach the goal with maximum cumulative reward in minimum steps.	Move towards goal while avoiding obstacles.
(2) Robotic Arm (Reaching)	Joint angles/positions of the arm. 	Continuous torques applied to each joint. 	+1 for reaching target position −1 for large error/collision	Move end-effector to target position accurately.	Apply torques to minimize distance to target.
(3) Stock Trading	Market indicators (price, volume, RSI, cash, holdings, etc.)	Buy, Sell, Hold	Change in portfolio value (profit/loss) after transaction cost.	Maximize total profit over time.	Buy low, sell high based on learned strategy.
(4) Personalized Recommendation	User features, item features, interaction history.	Recommend item i from catalog.	+1 if user clicks/buys, 0 otherwise.	Maximize long-term user engagement (clicks/likes/purchases).	Recommend items most likely to engage user.

Note: The agent learns a policy through interaction and feedback, improving its decisions over time to achieve the best possible outcome. ☺

Finite Markov Decision Processes (MDPs)

1. What is an MDP?

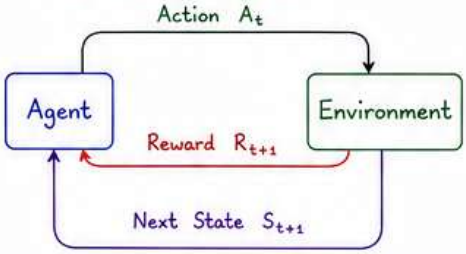
A Markov Decision Process (MDP) is a mathematical framework for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision maker (agent). It is defined by the tuple (S, A, P, R, γ) where

- S : Finite set of states.
- A : Finite set of actions.
- P : State transition probability function $P(s' \mid s, a) = \Pr\{S_{t+1} = s' \mid S_t = s, A_t = a\}$.
- R : Reward function $R(s, a)$ = expected immediate reward after taking action a in state s.
- γ : Discount factor, $0 \le \gamma < 1$ (future rewards are discounted by γ).

Key Property (Markov Property)

The next state depends only on the current state and action, not on the past history.

MDP Interaction Cycle



2. Objective

Find a policy $\pi : S \to A$ (mapping from states to actions) that maximizes the expected cumulative discounted reward:

$$J(\pi) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right]$$

Types of Policies

- Deterministic : a single action in each state.
- Stochastic : probability distribution over actions.
- Optimal policy π^* gives maximum $J(\pi)$.

3. Example 1: Grid World (Finite MDP)

A robot moves in a 2×2 grid. Goal is to reach the goal state G. Actions: Up (U), Down (D), Left (L), Right (R). Hitting a wall leaves the robot in the same state.

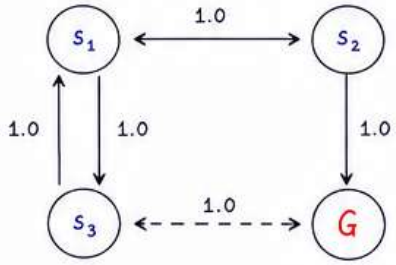
States:

S_1	S_2
S_3	G

Rewards:

- Reaching G : +10
- Each move : −1
- Hitting wall : −1

State Transition Diagram (action R is shown)



Example Episode (following policy π)

$S_1 \xrightarrow[-1]{R} S_2 \xrightarrow[-1]{D} G \quad (+10)$

Return $= -1 + \gamma(10)$

($\gamma = 0.9 \to$ Return $= -1 + 9 = 8$)

4. Example 2: Simple Coffee Machine MDP

A coffee machine can be in two states and we can choose actions Replenish (R) or Do nothing (N).

States:

$S = \{$ Full (F), Empty (E)$\}$

Actions:

$A = \{ R, N \}$

Rewards:

- Serve coffee in state F : +5
- Replenish in state E : −2
- Doing nothing in E : −1
- Doing nothing in F : 0
- Discount factor $\gamma = 0.9$

Transition Probabilities:

From state		To F	To E
F	R	1.0	0
	N	0.5	0.5
E	R	1.0	0
	N	0	1.0

This MDP is finite since it has finite states and actions.

The agent must learn when to replenish and when to wait, to maximize long-term coffee profits.

5. Why MDPs are Useful?

- Provide a clear framework for sequential decision-making under uncertainty.
- Applicable to robotics, games, resource management, scheduling, recommendation systems, etc.
- Many RL algorithms (value iteration, policy iteration, Q-learning) are built on MDPs.

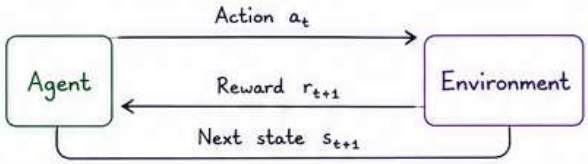
In short: A Finite MDP $= (S, A, P, R, \gamma)$ models an agent interacting with an environment over discrete time, aiming to learn an optimal policy that maximizes cumulative discounted reward. ☺

Finite Markov Decision Processes (MDPs)

1. Agent-Environment Interface

At each discrete time step t,

- Agent observes state $s_t \in S$
- Agent selects an action $a_t \in A$
- Environment transitions to next state $s_{t+1} \sim P(\cdot \mid s_t, a_t)$
- Environment gives reward $r_{t+1} = R(s_t, a_t)$



2. Goal and Rewards

The agent's goal is to learn a policy that maximizes the expected cumulative (discounted) reward over time.

Rewards:

- Positive reward : Good / desirable outcomes
- Negative reward : Penalty / undesirable outcomes
- Zero reward : Neutral outcome

Example:

- Reaching goal state : +10
- Each step cost : -1
- Hitting obstacle : -5

3. Returns and Episodes

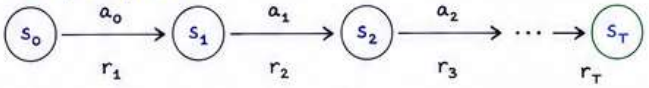
Return G_t from time step t is the cumulative discounted reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $0 \le \gamma < 1$ is the discount factor.

Episode: A sequence of states, actions and rewards from start to terminal state. (Episodes make the problem finite-horizon.)

Example Episode:



Episode ends at terminal state s_T.

4. Task: Episodic vs Continuous

Episodic Task	Continuous (Ongoing) Task
• Each episode has a beginning and an end.	• No terminal state; interaction continues indefinitely.
• The agent's experience is divided into episodes.	• Goal is to maximize long-term average reward.
• Example: Game of chess, Grid world (reach goal).	• Example: Robot navigation, Stock trading, Resource management.

5. Policies

A policy π is a mapping from states to actions.

- Deterministic policy: $\pi(s) = a$ (choose one action in each state)
- Stochastic policy: $\pi(a \mid s)$ (probability of taking action a in state s)

Goal: Find optimal policy π^* that maximizes expected return:

$$\pi^* = \arg\max_{\pi} E_\pi [G_t]$$

6. Value Function

Value function of a state under policy π:

$$V^\pi(s) = E_\pi [G_t \mid s_t = s] \quad \text{(State Value Function)}$$

Action-value function (Q-function):

$$Q^\pi(s, a) = E_\pi [G_t \mid s_t = s, a_t = a]$$

Interpretation:

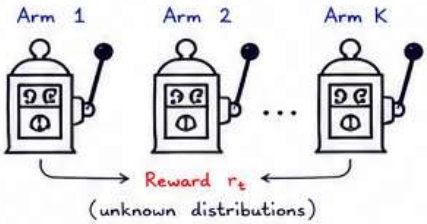
$V^\pi(s)$ is the expected return starting from state s and following policy π thereafter.
$Q^\pi(s, a)$ is the expected return starting from state s, taking action a, then following policy π.

7. Examples of Finite MDPs

Example	States (S)	Actions (A)	Rewards (R)	Goal
Grid World Navigation 	All cells in the grid (including obstacles if modeled).	Up, Down, Left, Right	+10 for reaching goal -1 per step -5 for hitting obstacle	Reach goal with maximum cumulative reward.
Robot Arm (Reaching) 	Joint angles or positions.	Apply torque (continuous or discrete set).	+1 for close to target -1 otherwise	Move end-effector to target position.
Resource Management	Resource levels (low, medium, high).	Allocate / Save / Use resource.	+R for good allocation -C for cost / shortage	Maximize long-term resource utility.

8. Multi-arm Bandit Introduction

A special case of MDP with only one state. The agent repeatedly chooses one of K actions (arms). Each action gives a random reward drawn from a fixed but unknown distribution.
Goal: Identify the best arm (action) that maximizes long-term reward.



Example:

Online ad selection, A/B testing, Recommendation, Clinical trials, Selecting best investment option.

FINITE DECISION PROCESS (FDP) – SOLVED PROBLEM

What is FDP?

An episodic task with finite states, actions and time steps (finite horizon). The agent aims to maximize total expected reward.

Notation

- S_t : state at time t
- a_t : action at time t
- R_{t+1} : reward received after taking a_t in s_t
- T : terminal time step (episode ends at $t = T$)
- γ : discount factor $(0 \le \gamma \le 1)$
- π : policy

Goal

Find optimal policy π^* that maximizes expected return from any state.

General Update (FDP / Finite Horizon DP)

$$V_t(s) = \max \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V_{t+1}(s') \right]$$

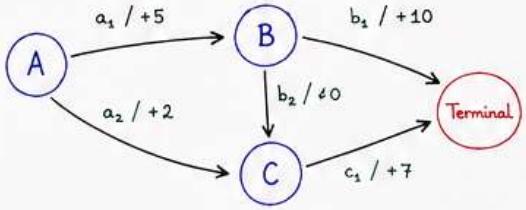
- $P(s' \mid s, a)$: Probability of going to s'
- $R(s, a, s')$: Immediate reward
- $V_{t+1}(s')$: Future value

How to solve an FDP? (Backward Induction)

1. At terminal time T, $V_T(s) = 0$ for all states.
2. For $t = T-1$ down to 0 :
   For each state s :
   $V_t(s) = \max_a \left[r(s, a) + \gamma V_{t+1}(s') \right]$
   $a^*(s)$ = action that gives the max value.
3. Optimal policy $\pi^*(s) = a^*(s)$ for all states.

PROBLEM 1 : Consider the following FDP with horizon $T = 3$ (time steps 0, 1, 2 and terminal at 3). Discount factor $\gamma = 1$. Rewards are on transitions. Find the optimal policy and maximum expected return from each state.

State Transition Diagram



a_1, a_2 available in A; b_1, b_2 in B; c_1 in C.

Step 0: Terminal time $t = T = 3$

$V_3(\text{Terminal}) = 0$

We solve by working backward from the end to the start!

Step 1: Compute values at $t = 2$ (one step before terminal)

State	Action	Next State	Reward	Value of Next State $V_3(s')$	Total Return $r + \gamma V_3(s')$	$V_2(s)$ max over actions
B	b_1	Terminal	+10	0	$10 + 1 \times 0 = 10$	10 (choose b_1)
	b_2	C	0	0	$0 + 1 \times 0 = 0$	
C	c_1	Terminal	+7	0	$7 + 1 \times 0 = 7$	7 (choose c_1)

At $t = 2$, only one step is left to reach terminal. So we just pick the action with highest immediate reward.

Step 2: Compute values at $t = 1$

State	Action	Next State	Reward	Value of Next State $V_2(s')$	Total Return $r + \gamma V_2(s')$	$V_1(s)$ max over actions
A	a_1	B	+5	$V_2(B) = 10$	$5 + 1 \times 10 = 15$	15 (choose a_1)
	a_2	C	+2	$V_2(C) = 7$	$2 + 1 \times 7 = 9$	

From A, going to B is better because it leads to higher future value.

Step 3: Compute values at $t = 0$ (initial decision time)

State	Action	Next State	Reward	Value of Next State $V_1(s')$	Total Return $r + \gamma V_1(s')$	$V_0(s)$ max over actions
A	a_1	B	+5	$V_1(B) = 10$	$5 + 1 \times 10 = 15$	15 (choose a_1)
	a_2	C	+2	$V_1(C) = 7$	$2 + 1 \times 7 = 9$	

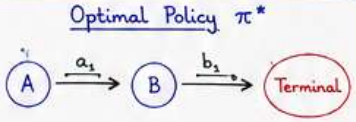
Optimal Value Function

$V_0(A) = 15$
$V_1(A) = 15$
$V_2(B) = 10$
$V_2(C) = 7$
$V_3(\text{Terminal}) = 0$

Step 4: Derive Optimal Policy

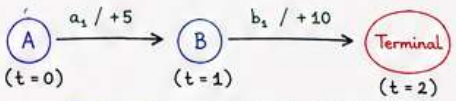
$t = 0$ at A → choose a_1 → go to B
$t = 1$ at B → choose b_1 → go to Terminal
(Reaching terminal at $t = 2$)

Optimal Policy π^*



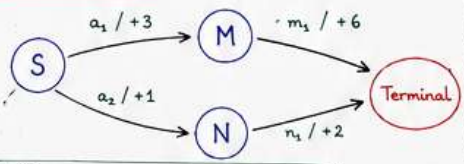
(Total expected return = 15)

Path followed by π^*



Total return = 5 + 10 = 15

PROBLEM 2 : Another FDP



Given:
- Horizon $T = 2$ (0, 1 and terminal at 2)
- $\gamma = 1$
- Start state = S

Step 0: $t = 2$ (terminal)

$V_2(\text{Terminal}) = 0$

Step 1: $t = 1$

State	Action	Next	Reward	$V_1(s)$
M	m_1	Terminal	+6	6
N	n_1	Terminal	+2	2

Step 2: $t = 0$ (initial)

State	Action	Next	Reward	$V_1(s')$	Total	$V_0(s)$
S	a_1	M	+3	6	3+6=9	9 (choose a_1)
	a_2	N	+1	2	1+2=3	

Answer:
Optimal policy : At S choose a_1 then m_1
Maximum Expected Return from S = 9

KEY TAKEAWAYS

- ✓ FDP uses backward induction (finite horizon DP).
- ✓ At each state and time, pick action that maximizes $r + \gamma \times$ (future value).
- ✓ By working backward, we get optimal value function $V_t(s)$ and optimal policy π^*.

ALGORITHM SUMMARY

1. Set $V_T(s) = 0$ for all terminal states.
2. For $t = T-1$ down to 0 :
   For each state s : compute $V_t(s)$ using the update rule.
3. Record the action $a^*(s)$ that maximizes the value.
4. The policy π^* is $\{a^*(s)\}$ for all states s.

REMEMBER

FDP (finite horizon)
→ solved by DP (backward)
MDP (infinite horizon)
→ solved by value iteration / policy iteration / RL.

Dynamic Programming Methods in Reinforcement Learning

① Introduction

What is Dynamic Programming?

- Dynamic Programming (DP) is a collection of methods used to solve Markov Decision Processes (MDPs) when the model of the environment is known.
- DP methods help an RL agent to:
  - Evaluate a policy
  - Improve a policy
  - Find the optimal policy

Requirements of DP

- State transition probabilities $P(s'|s,a)$
- Reward function $R(s,a,s')$
- Finite state and action spaces

Key Idea

- Break a complex decision-making problem into smaller subproblems and solve them recursively.

Example: Grid World

S	→	→
↓		
←	←	G

Rewards:
- +10 for reaching Goal (G)
- −1 for every move

Objective:
- Find the shortest path to maximize reward.

② Policy Evaluation

Definition

Policy Evaluation computes the value function $V^\pi(s)$ for a given policy π.
It answers: "How good is this policy?"

Bellman Expectation Equation

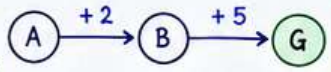
$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V^\pi(s')\right]$$

Algorithm (Iterative policy evaluation)

① Initialize all state values to zero (or any value).
② Update values using the Bellman equation.
③ Repeat until convergence ($|V_{new} - V_{old}| < \varepsilon$).

Example:

Policy: A → B → Goal
Rewards: A → B = +2 B → Goal = +5
Discount factor: $\gamma = 1$



Iteration 1:
- V(G) = 0
- V(B) = 5
- V(A) = 2

Iteration 2:
- V(A) = 2 + V(B) = 2 + 5 = 7

Final values:
- V(A) = 7, V(B) = 5, V(G) = 0

③ Policy Improvement

Definition

Policy Improvement improves an existing policy using the value function obtained from policy evaluation.

Key Idea

- Choose the action with the highest expected return.

Bellman Optimality Principle

$$\pi'(s) = \arg\max_a \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V^\pi(s')\right]$$

Example:

State A has two actions:
- Action 1 → Reward = 3
- Action 2 → Reward = 7

Current policy chooses Action 1.
Since $7 > 3$, improved policy chooses Action 2.

Policy Improvement Theorem

If $Q^\pi(s,\pi'(s)) \geq V^\pi(s)$
then
$V^{\pi'}(s) \geq V^\pi(s)$.

The new policy is always better or equal.

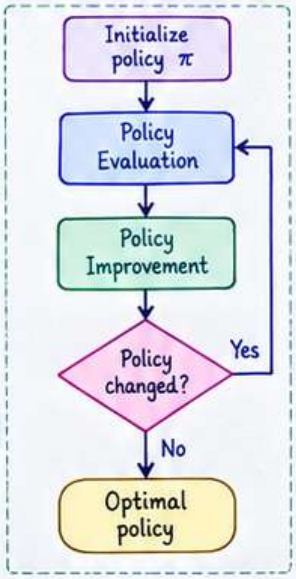
④ Policy Iteration

Idea

- Repeatedly perform:
  ① Policy Evaluation → compute $V^\pi(s)$
  ② Policy Improvement → improve the policy
  until the policy stops changing.

Algorithm

① Initialize policy π
② Evaluate policy → compute $V^\pi(s)$
③ Improve policy → $\pi'(s) = \arg\max_a \sum_{s'} P(s'|s,a)\left[R + \gamma V^\pi(s')\right]$
④ If policy changes, repeat from step 2. Otherwise, stop. (π' is optimal)



⑤ Value Iteration

Idea

- Combines policy evaluation and policy improvement into a single update step.

Bellman Optimality Equation

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V_k(s')\right]$$

Algorithm

① Initialize V(s) = 0
② Apply Bellman optimality update
③ Repeat until convergence
④ Extract policy from final values

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s'|s,a)\left[R + \gamma V^*(s')\right]$$

Example

State A has two actions:
- Action 1 → Reward = 2
- Action 2 → Reward = 5

Initial: V(A) = 0
Update: V(A) = max(2, 5) = 5
Next iteration: $V(A) = 5 + \gamma V(\text{next}) \ldots$
Continue until convergence.

⑥ Policy Iteration vs Value Iteration

Feature	Policy Iteration	Value Iteration
Uses Policy Evaluation	Yes	No
Uses Policy Improvement	Yes	Implicit
Convergence Speed	Fewer iterations	More iterations
Computation per Iteration	Higher	Lower
Memory Requirement	Higher	Lower
Produces Optimal Policy	Yes	Yes

Key Takeaways

- Policy Evaluation computes state values for a given policy.
- Policy Improvement creates a better policy.
- Policy Iteration alternates evaluation and improvement.
- Value Iteration directly applies Bellman optimality updates.
- Both methods find the optimal policy in finite MDPs.
- Value Iteration is generally preferred for large state spaces.

Remember:

"Policy Evaluation → How good is the current policy?"
"Policy Improvement: → Can we make it better?"
"Policy Iteration → Evaluation + Improvement repeatedly."
"Value Iteration → Directly find optimal values."

Better policies → Better decisions → Higher rewards!